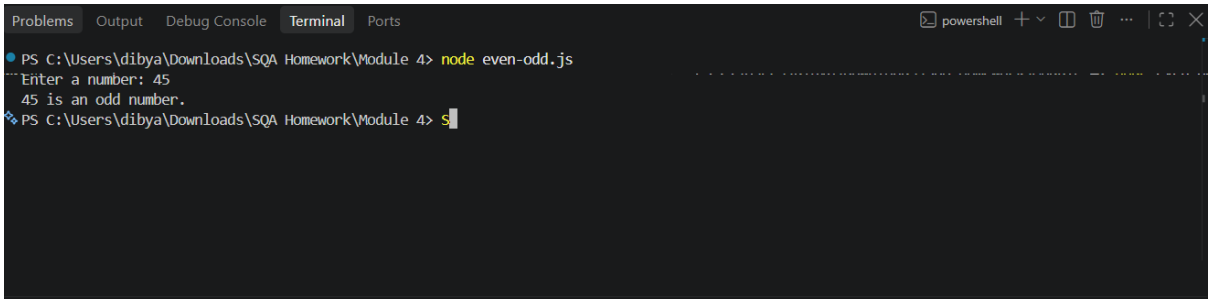
Name: Dibya Jyoti Dhar

Assignment 4

Q1: JavaScript Coding Practice

i. Check whether a number is even or odd.

Output

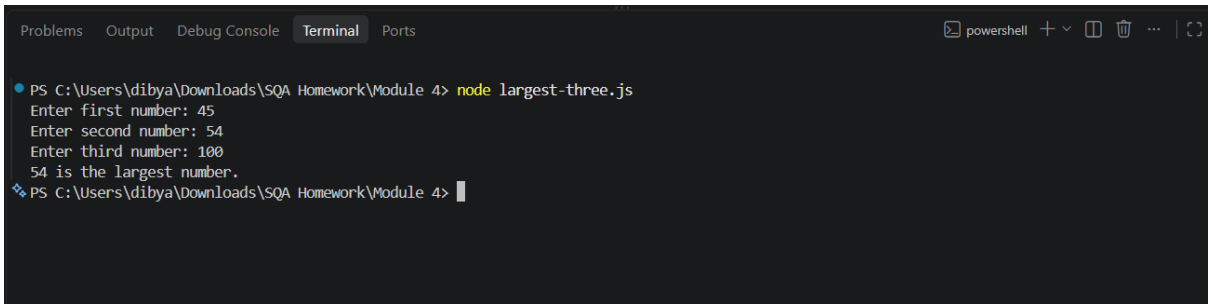


Explanation

This program takes a number from the user and checks whether it is even or odd. The `readline` module is used to receive input from the terminal. The `%` operator finds the remainder when the number is divided by 2. If the remainder is `0`, the number is even; otherwise, it is odd. `rl.close()` closes the input interface after the program finishes.

ii. Find the largest of three numbers.

Output

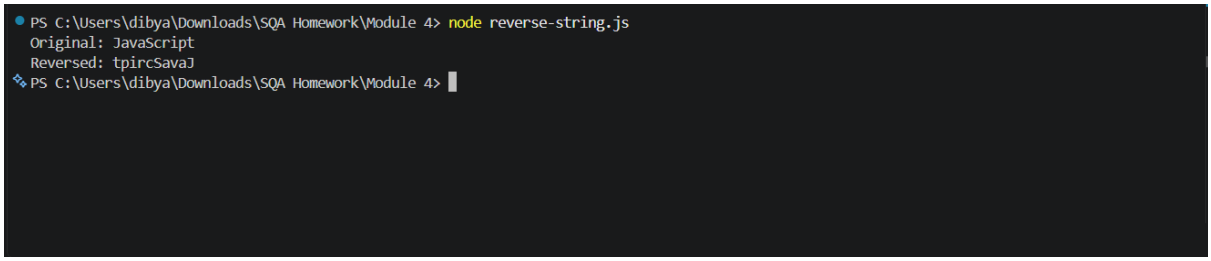


Explanation

This program takes three numbers as input from the user and compares them to find the largest one. The `if...else if...else` statements compare each number with the other two using >= and &&. The largest number is then displayed using `console.log()`. Finally, `rl.close()` closes the input interface.

iii. Reverse a string.

Output

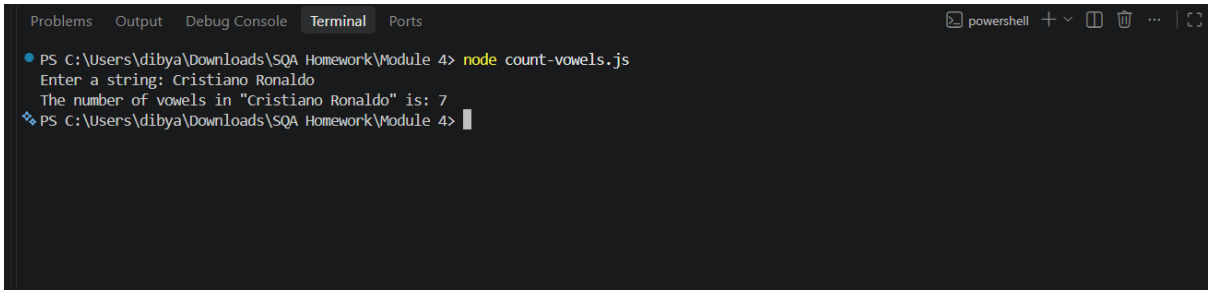


Explanation

This program reverses the string `"JavaScript"`. First, `split("")` converts the string into an array of individual characters. Then, `reverse()` reverses the array, and `join("")` combines the characters back into a string. Finally, `console.log()` displays the original and reversed strings.

iv. Count vowels in a string.

Output



Explanation

This program takes a string as input and counts how many vowels it contains.
The `for...of` loop checks each character in the string. The `includes()` method checks whether the character is a vowel (`a, e, i, o, u`) in either uppercase or lowercase. Each time a vowel is found, `vowelsCount` is increased by 1. Finally, the total number of vowels is displayed, and `rl.close()` closes the input interface.

v. Remove duplicate values from an array.

Output

Explanation

This program takes numbers as input, separates them into an array using `split(' ')`, and removes duplicate values using `Set`. The spread operator `...` converts the `Set` back into an array. Finally, it displays both the original array and the array without duplicates. `rl.close()` closes the input interface.

Q2. Introduction to Node.js

1. What is Node.js?

Node.js is a runtime environment that allows JavaScript to run outside a web browser. It is built on Google's V8 JavaScript engine.

Normally, JavaScript runs inside browsers such as Chrome or Edge and is mainly used for web page interaction. With Node.js, JavaScript can also be used for backend development, automation, API testing, file handling, and command-line applications.

2. How Does Node.js Work?

Node.js uses an event-driven and non-blocking architecture. It uses an event loop to handle multiple operations efficiently.

For example, when Node.js needs to perform an operation such as reading a file or making a network request, it can continue working on other tasks instead of waiting for that operation to finish.

For example:

```
console.log("Start");

setTimeout(() => {

    console.log("Task completed");

}, 2000);
```

```
console.log("End");
```

Output:

Start

End

Task completed

The program doesn't stop completely while waiting for the timer. Node.js continues executing other code and handles the completed task later.

3. Advantages of Node.js for QA Automation

Node.js is very useful for QA automation because many popular testing tools use the Node.js ecosystem.

Major advantages:

i. Web Automation: Node.js supports popular automation frameworks such as Cypress, Playwright, and WebdriverIO.

ii. API Testing: QA engineers can use Node.js libraries and frameworks to send API requests and verify:

- Status codes
- Response data
- Headers
- Response time

iii. Fast and Efficient: Its asynchronous and event-driven architecture is useful for handling multiple automation operations efficiently.

iv. Easy CI/CD Integration: Node.js-based tests can easily be integrated with tools such as Jenkins, GitHub Actions, and GitLab CI/CD.

v. Large `npm` Ecosystem: Node.js comes with **npm (Node Package Manager)**, which provides thousands of packages for testing, automation, reporting, API testing, and other QA activities.

vi. JavaScript/TypeScript Support: QA engineers can use JavaScript or TypeScript to write automated tests, making it easier to work with modern web applications.

4. Difference Between Browser JavaScript and Node.js

Feature	Browser JavaScript	Node.js
Where it runs	Web browser	Computer/server
Main purpose	Web page interaction	Backend, automation, scripts
DOM access	Yes	No, by default
window object	Available	Not available
File system access	Restricted	Available
Database access	Usually through APIs	Can use database libraries
npm packages	Not directly	Yes
HTTP server	Not normally	Yes
QA Automation	Runs mainly through browser tools	Used to run automation frameworks

Simple Example

Browser JavaScript can interact directly with an HTML element:

```
document.getElementById("login");
```

Node.js doesn't have a browser DOM by default, so document isn't available.

```
On the other hand, Node.js can access the file system:
```

```
const fs = require("fs");
```

```
fs.writeFileSync("test.txt", "Hello Node.js");
```

GitHub for review: https://github.com/dibya888/Ostad-SQA-Assignment/tree/master/Module%204